

Module 12: Collections

Four shapes, several implementations each. Choosing correctly is most of the skill, and the rest is knowing which three or four traps exist.

Two of those traps are the same rule you have already met twice. Module 06 lost an object inside a `HashSet`; Module 08 warned about a mutable component in a record. Both were this module's rule, seen from different angles.

What You'll Learn

- `List`, `Set`, `Map`, `Deque`, and which implementation to reach for
- Sequenced collections, and the thirty-year gap they closed
- `List.of` and friends, with four behaviours that surprise people
- Why removing during iteration throws, and the four correct alternatives
- The rule about keys, stated plainly at last

Coming From Another Language

	Python	Java
Ordered sequence	<code>list</code>	<code>ArrayList</code>
Unique	<code>set</code>	<code>HashSet</code> , or <code>LinkedHashSet</code> for order
Key to value	<code>dict</code> , insertion-ordered	<code>HashMap</code> unordered, <code>LinkedHashMap</code> ordered
Sorted	<code>sorted()</code> on demand	<code>TreeMap</code> , <code>TreeSet</code> keep it sorted
Immutable literal	<code>tuple</code> , <code>frozenset</code>	<code>List.of</code> , <code>Set.of</code> , <code>Map.of</code>
Stack / queue	<code>list</code> , <code>deque</code>	<code>ArrayDeque</code>

The row that catches Python developers: **`HashMap` does not preserve insertion order.** Python dictionaries have since 3.7. If you need the order, say so by using `LinkedHashMap`.

The Lesson

Choosing

From `ChoosingACollection.java`:

Need	Use
index, or duplicates	<code>ArrayList</code>
uniqueness, order irrelevant	<code>HashSet</code>
uniqueness, stable output	<code>LinkedHashSet</code>
sorted	<code>TreeSet</code> / <code>TreeMap</code>
key to value	<code>HashMap</code> , or <code>LinkedHashMap</code>
stack or queue	<code>ArrayDeque</code>

`ArrayList` versus `LinkedList` : use `ArrayList` . A linked list gives $O(1)$ insertion *at a known position*, but finding that position is $O(n)$, and every element costs an extra object plus two pointers. Sequential memory is fast and pointer chasing is not, so `ArrayList` wins in practice even where the big-O table says it should lose.

Never use `java.util.Stack` . It extends `Vector` , synchronises every method for a threading model abandoned in 1998, and iterates bottom-to-top, which is backwards for a stack. `ArrayDeque` is the answer for both stacks and queues.

A `HashSet` 's iteration order is not a promise. It looking stable in your tests means nothing; it can change between JDK versions.

Three `Map` methods worth learning immediately:

```
map.getOrDefault(key, 0)
map.computeIfAbsent(key, k -> new ArrayList<>()).add(value)    // map of lists
map.merge(word, 1, Integer::sum)                               // counting
```

Sequenced collections

Added in Java 21 to close a gap that had been open since 1998. Getting the first element used to be three different expressions depending on the type, and getting the **last** element of a `LinkedHashSet` meant iterating the whole thing.

From `SequencedCollections.java` :

```
getFirst:  a
getLast:   c
reversed:  [c, b, a]
after add, the view shows: [d, c, b, a]
```

`reversed()` is a **view**, not a copy. It costs nothing to create and reflects later changes to the original.

`SequencedMap` adds `firstEntry` , `lastEntry` , `putFirst` and `putLast` :

```
firstEntry: x=1
putFirst:   {w=0, x=1, y=2, z=3}
```

Interface	Implemented by
SequencedCollection	List , Deque , LinkedHashSet , SortedSet
SequencedSet	LinkedHashSet , TreeSet
SequencedMap	LinkedHashMap , TreeMap

HashSet and HashMap are deliberately **not** sequenced. They have no defined order, so the compiler now refuses the question rather than letting you iterate and hope.

The immutable factories

From `ImmutableFactories.java`, four surprises, all deliberate:

1. **Genuinely immutable.** Every mutating method throws. Not "unmodifiable by convention".
2. **null is rejected.** `List.of("a", null)` throws `NullPointerException`. Converting old code to `List.of` can surface nulls you did not know you had.
3. **Duplicates are an error.** `Set.of(1, 1)` throws `duplicate element: 1`, where `new HashSet<>(List.of(1, 1))` would quietly give you one element. In a literal, a duplicate is almost always a typo.
4. **Set.of iteration order is unspecified** and deliberately varies. If you need order, use `List.of` or `LinkedHashSet`.

Three things that look alike and are not:

	add	set
<code>List.of</code>	throws	throws
<code>Arrays.asList</code>	throws	ok
<code>Collections.unmodifiableList</code>	throws	throws

`Arrays.asList` is **fixed size, not immutable**, and writes through to the backing array.

`Collections.unmodifiableList` is a **view** over a mutable list:

```
after mutating the source:
  unmodifiable VIEW sees it: [x, y]
  List.copyOf does not:     [x]
```

`List.copyOf` is usually what you want, which is why Modules 05 and 08 both reached for it.

Mutation during iteration

`MutationDuringIteration.java` throws:

```
Exception in thread "main" java.util.ConcurrentModificationException
  at java.base/java.util.ArrayList$Itr.checkForComodification
```

"Concurrent" is misleading. There is one thread. It means the collection changed concurrently with an iteration, not from another thread.

`ArrayList` keeps a `modCount` that increments on every structural change. The iterator records it and checks before every step, failing fast rather than skipping elements or reading past the end. It is documented as **best-effort**, so never write code that relies on the exception being thrown.

Four correct alternatives:

```
numbers.removeIf(n -> n % 2 == 0);           // 1. usually this

var it = numbers.iterator();                  // 2. when a predicate is not enough
while (it.hasNext()) {
    if (it.next() % 2 == 0) it.remove();
}

// 3. collect, then remove after
// 4. build a new collection (Module 14)
```

The rule about keys

`KeysMustNotChange.java` puts a mutable object in a map and renames it:

```
after renaming the key object:
get(key):      null
containsKey:   false
size:          1
but iterating still finds it: [MutableKey(invoices)]
after remove(key), size is still: 1
```

The entry sits in the bucket for its **old** hash code. Lookup computes the new one, searches a different bucket, and finds nothing. The map still counts it and iteration still yields it, so the data is not gone. It is merely unreachable by lookup, which is the one thing a map exists to do. It is also now unremovable by key.

Map keys and set elements must be immutable in every field that `equals` and `hashCode` use.

A record whose components are themselves immutable satisfies this for free. That is most of why records and maps go together so well, and it is Module 08's warning restated from the other side.

Run It

```
java modules/12-collections/examples/ChoosingACollection.java
java modules/12-collections/examples/SequencedCollections.java
java modules/12-collections/examples/ImmutableFactories.java
java modules/12-collections/examples/KeysMustNotChange.java

# Fails on purpose.
java modules/12-collections/examples/MutationDuringIteration.java

./scripts/verify-examples.sh modules/12-collections
```

Common Mistakes

Depending on `HashMap` or `HashSet` **iteration order**. It is not defined and it does change.

Removing inside a for-each loop. Use `removeIf`.

Using a mutable object as a key. The entry becomes unreachable the moment a relevant field changes.

Expecting `Arrays.asList` to be immutable. It is fixed-size, and `set` writes through to the array behind it.

Returning `Collections.unmodifiableList(internalList)` and believing it is safe. It is a view. Whoever holds the original can still change what the caller sees.

Reaching for `LinkedList` because insertion is $O(1)$. Finding the insertion point is not, and the constant factors are bad.

Using `java.util.Stack`. Synchronised for no reason, and it iterates the wrong way round.

Key Takeaways

- `ArrayList`, `HashMap`, `ArrayDeque` **cover most needs**. Reach elsewhere for a stated reason.
- Use `LinkedHashSet` or `LinkedHashMap` **whenever order is observable**, such as in output or tests.
- **Sequenced collections** give `getFirst`, `getLast` and `reversed()`, and `reversed()` is a free view.
- `List.of` **rejects null, rejects duplicate set entries, and is truly immutable**. `Arrays.asList` and `unmodifiableList` are neither.
- `removeIf`, **not removal inside a loop**.
- **Keys must not change** in any field `equals` or `hashCode` reads.

Homework

[homework/README.md](#)

Build a word index, pick the right collection at each step, and make a map entry unreachable on purpose. Reference solution in [solutions/12-collections/](#).